

Somaiya Vidyavihar University
K. J. Somaiya School of Engineering, Mumbai-77

Batch: C3 Roll No.: 16010123325

Experiment / Assignment / Tutorial No. _4_

Grade: AA / AB / BB / BC / CC / CD /DD

Title: CAPTCHA

Objective:

The primary objective is to research, design, and programmatically implement various CAPTCHA systems used to differentiate human users from automated bots. Specifically, this involves exploring the underlying mechanics and building functional models of four distinct challenge types: distorted text entry, accessible audio codes, simple mathematical puzzles, and image-based object recognition grids.

Expected Outcome of Experiment:

Course Outcome	After successful completion of the course students should be able to
	Upon successful completion of this module, students will be able to independently design, code, and integrate multiple forms of CAPTCHA systems into web applications to defend against automated bots. Furthermore, they will possess the practical skills to evaluate and select the appropriate verification mechanism for a given scenario, effectively balancing robust web security with user accessibility and a seamless user experience

Books/ Journals/ Websites referred:

<https://www.youtube.com/watch?v=bfKwizfuuOU>
<https://www.cloudflare.com/learning/bots/how-captchas-work/>
<https://www.fortinet.com/uk/resources/cyberglossary/captcha>

Somaiya Vidyavihar University
K. J. Somaiya School of Engineering, Mumbai-77

Pre Lab/ Prior Concepts:

Web Technologies: Familiarity with HTML, CSS, and basic JavaScript for building web forms and interactive elements.

Client-Server Architecture: Understanding how browsers communicate with web servers via HTTP/HTTPS requests.

Bots & Automation: Basic awareness of automated scripts/bots that can interact with web applications programmatically.

Human-Computer Interaction (HCI): Understanding how users interact with web-based forms and UI elements.

Basic Cryptography & Randomness: Awareness of random string/number generation used in challenges.

New Concepts to be learned:

CAPTCHA – What it is, its full form (Completely Automated Public Turing test to tell Computers and Humans Apart), and its purpose in web security.

Types of CAPTCHA: Text-based, Audio-based, Mathematical Puzzle, Image-based (Grid / Object Recognition), and Invisible/Behavioral CAPTCHA.

reCAPTCHA: Google's evolution of CAPTCHA (v1, v2, v3) and how it uses risk analysis.

Bot Detection Mechanisms: How CAPTCHAs distinguish human users from automated bots.

Accessibility Considerations: Why audio-based CAPTCHAs exist and their importance for visually impaired users.

Abstract:

CAPTCHA (Completely Automated Public Turing test to tell Computers and Humans Apart) is a widely used security mechanism designed to differentiate between human users and automated bots on web platforms. This experiment explores the concept, working principles, and various types of CAPTCHAs employed in modern web security. Through hands-on implementation, students will design and develop four major forms of CAPTCHAs text-based, audio-based, mathematical puzzle, and image-based (grid/object recognition) using Python and web technologies. The experiment aims to provide a deep understanding of how CAPTCHAs protect online services from spam, credential stuffing, brute-force attacks, and other automated threats, while also highlighting the trade-off between security robustness and user accessibility.

Somaiya Vidyavihar University

K. J. Somaiya School of Engineering, Mumbai-77

Related Theory:

CAPTCHA stands for **Completely Automated Public Turing test to tell Computers and Humans Apart**. It is a type of challenge-response authentication mechanism used in computing to determine whether the user is a human or an automated bot. The concept is derived from the **Turing Test**, proposed by Alan Turing in 1950, which evaluates a machine's ability to exhibit intelligent behavior indistinguishable from a human.

CAPTCHAs are typically used to:

- Prevent automated form submissions and spam.
- Protect login pages from brute-force attacks.
- Secure online polls, ticket booking, and e-commerce checkouts.
- Block content scraping and account creation by bots.

Types of CAPTCHA:

a) Text-Based CAPTCHA

The most traditional form. A randomly generated string of alphanumeric characters is displayed with distortions, noise, and warping to make OCR (Optical Character Recognition) by bots difficult. The user must correctly type the displayed text.

b) Audio-Based CAPTCHA

Designed as an **accessibility alternative** for visually impaired users. A sequence of spoken letters or digits is played, and the user must type what they hear. Background noise is added to confuse automated audio recognition systems.

c) Mathematical Puzzle CAPTCHA

The user is presented with a simple arithmetic problem (e.g., "What is $7 + 4$?"). While trivial for humans, it acts as a deterrent for simple bots that do not parse natural language or arithmetic expressions.

d) Image-Based CAPTCHA (Grid / Object Recognition)

The user is shown a 3×3 or 4×4 image grid and asked to select all tiles containing a specific object (e.g., "Select all images with traffic lights"). This leverages human visual perception, which was historically hard for machines to replicate.

e) Behavioral / Invisible CAPTCHA (reCAPTCHA v3)

No visible challenge is presented. Instead, user behavior (mouse movement patterns, typing speed, scrolling, time on page) is analyzed in the background to assign a risk score. Low-score interactions are flagged as bots.

Somaiya Vidyavihar University K. J. Somaiya School of Engineering, Mumbai-77

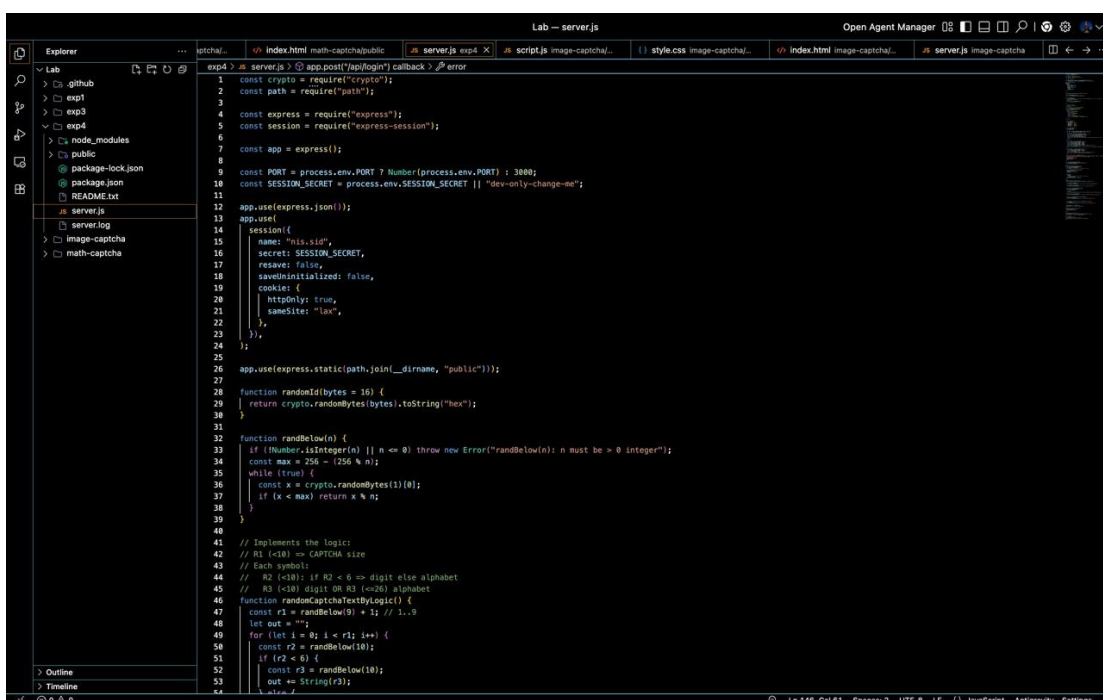
Implementation Details:

The CAPTCHA system follows a **client-server architectural model** where the frontend and backend work together to generate and validate challenges.

On the **server side**, Node.js powers the backend, which is responsible for dynamically producing CAPTCHA challenges through randomized logic. It exposes dedicated API endpoints namely /captcha and /verify to handle challenge generation and response validation respectively, ensuring that all input verification takes place securely at the server level.

On the **client side**, the interface is built using HTML, CSS, and JavaScript. It communicates with the backend via the fetch() API to retrieve freshly generated CAPTCHAs, renders them in a user-friendly format, collects the user's response, and forwards it to the backend endpoint for validation.

Text-Based



```

1  const crypto = require('crypto');
2  const path = require('path');
3
4  const express = require('express');
5  const session = require('express-session');
6
7  const app = express();
8
9  const PORT = process.env.PORT ? Number(process.env.PORT) : 3000;
10 const SESSION_SECRET = process.env.SESSION_SECRET || "dev-only-change-me";
11
12 app.use(express.json());
13 app.use(
14   session({
15     name: "sess.sid",
16     secret: SESSION_SECRET,
17     resave: false,
18     saveUninitialized: false,
19     cookies: {
20       httpOnly: true,
21       sameSite: "lax",
22     },
23   })
24 );
25
26 app.use(express.static(path.join(__dirname, "public")));
27
28 function randomId(bytes = 36) {
29   return crypto.randomBytes(bytes).toString("hex");
30 }
31
32 function randBelow(n) {
33   if (!Number.isInteger(n) || n <= 0) throw new Error("randBelow(n): n must be > 0 integer");
34   const max = 256 - (256 % n);
35   while (true) {
36     const x = crypto.randomBytes(1)[0];
37     if (x < max) return x % n;
38   }
39 }
40
41 // Implements the logic:
42 // R1 (<10) => CAPTCHA size
43 // Each symbol:
44 // R2 (<10): if R2 < 6 => digit else alphabet
45 // R3 (<10) digit OR R3 (<26) alphabet
46 function randomCapchaTextByLogic() {
47   const r1 = randBelow(9) + 1; // 1..9
48   let out = "";
49   for (let i = 0; i < r1; i++) {
50     const r2 = randBelow(10);
51     if (r2 < 6) {
52       const r3 = randBelow(10);
53       out += String(r3);
54     } else {
55       const r3 = randBelow(26);
56       out += String.fromCharCode(65 + r3);
57     }
58   }
59   return out;
60 }
  
```

Somaiya Vidyavihar University

K. J. Somaiya School of Engineering, Mumbai-77

Image Grid

```

Lab - server.js
Open Agent Manager

Explorer
  Lab
    > .github
    > > exp1
    > > exp3
    > > exp4
    > image-captcha
      > node_modules
      > public
      > package-lock.json
      > package.json
      > server.js
      > math-captcha

  > Outline
  > Timeline

  0 0 0

  1 const express = require('express');
  2 const cors = require('cors');
  3 const bodyParser = require('body-parser');
  4 const path = require('path');
  5
  6 const app = express();
  7 const port = 4000;
  8
  9 app.use(cors());
 10 app.use(bodyParser.json());
 11 app.use(express.static(path.join(__dirname, 'public')));
 12
 13 // Store captchas in memory
 14 const captchas = {};
 15
 16 function generateId() {
 17   return Math.random().toString(36).substr(2, 9);
 18 }
 19
 20 // Real-world objects for image categories
 21 const categories = ['car', 'bicycle', 'bus', 'cat'];
 22
 23 function getImageTag(category) {
 24   const randomLock = Math.floor(Math.random() * 10000);
 25   const url = `https://loremflickr.com/200/200/s(category)?lock=${randomLock}`;
 26
 27   return ``;
 28 }
 29
 30 app.get('/api/captcha', (req, res) => {
 31   // Pick a random target category (e.g., 'car')
 32   const targetCategory = categories[Math.floor(Math.random() * categories.length)];
 33
 34   const grid = [];
 35   const correctIndexes = [];
 36   let hasTarget = false;
 37
 38   for (let i = 0; i < 9; i++) {
 39     let isTarget = Math.random() > 0.6;
 40     // for each row
 41     if (i === 0 && !hasTarget) isTarget = true;
 42
 43     const actualCategory = isTarget ? targetCategory : categories.filter(c => c !== targetCategory)[Math.floor(Math.random() * 3)];
 44
 45     if (isTarget) {
 46       hasTarget = true;
 47       correctIndexes.push(i);
 48     }
 49
 50     grid.push({
 51       index: i,
 52       image: getImageTag(actualCategory)
 53     });
 54   }
 55 }
 56
 57 
```

Mathematical Puzzle Based

```

Lab - server.js
Open Agent Manager

Explorer
  Lab
    > .github
    > audio-captcha
      > node_modules
      > public
      > package-lock.json
      > package.json
      > server.js
      > math-captcha

  > Outline
  > Timeline

  0 0 0

  1 const express = require('express');
  2 const cors = require('cors');
  3 const bodyParser = require('body-parser');
  4 const path = require('path');
  5
  6 const app = express();
  7 const port = 5000; // Using a unique port 5000
  8
  9 app.use(cors());
 10 app.use(bodyParser.json());
 11 app.use(express.static(path.join(__dirname, 'public')));
 12
 13 // Store captchas in memory
 14 const captchas = {};
 15
 16 function generateId() {
 17   return Math.random().toString(36).substr(2, 9);
 18 }
 19
 20 function generateSpokenString() {
 21   const digits = [];
 22   for (let i = 0; i < 5; i++) {
 23     digits.push(Math.floor(Math.random() * 10));
 24   }
 25   return digits;
 26 }
 27
 28 app.get('/api/captcha', (req, res) => {
 29   const id = generateId();
 30   const digits = generateSpokenString();
 31
 32   // Store the raw string (e.g., "59483") for validation
 33   captchas[id] = digits.join('');
 34
 35   setTimeout(() => {
 36     delete captchas[id];
 37     // 5 * 60 * 1000;
 38   }, 5 * 60 * 1000);
 39
 40   // Return the ID so the frontend can then request the audio specifically when the button is pressed
 41   res.json({ id });
 42 });
 43
 44 app.get('/api/audio/:id', async (req, res) => {
 45   const id = req.params.id;
 46   if (!captchas.hasOwnProperty(id)) {
 47     return res.status(404).send('Audio not found or expired');
 48   }
 49
 50   // Space out the digits so Google TTS reads them one by one ("five . nine . four")
 51   const textToSpeak = captchas[id].split('').join(' . ');
 52   const googleTtsUrl = `https://translate.google.com/translate_tts?ie=UTF-8&client=tw-ob&gl=en&q=${encodeURIComponent(textToSpeak)}`;
 53   try {
 54     const response = await fetch(googleTtsUrl);
 55     const audio = await response.arrayBuffer();
 56     res.set('Content-Type', 'audio/mpeg');
 57     res.send(Buffer.from(audio));
 58   } catch (error) {
 59     console.error('Error fetching audio:', error);
 60     res.status(500).send('Internal server error');
 61   }
 62 }
 63
 64 
```

Somaiya Vidyavihar University K. J. Somaiya School of Engineering, Mumbai-77

Audio Based

```

Lab - server.js
Open Agent Manager

math-captcha > js server.js >
1 const express = require('express');
2 const cors = require('cors');
3 const bodyParser = require('body-parser');
4 const path = require('path');
5
6 const app = express();
7 const port = 3000;
8
9 app.use(cors());
10 app.use(bodyParser.json());
11 app.get('/', (req, res) => {
12   res.sendFile(path.join(__dirname, 'index.html'));
13 });
14
15 // Store captchas in memory.
16 // For a production app, use Redis or a database.
17 const captchas = {};
18
19 // Helper function to generate a random ID
20 function generateId() {
21   return Math.random().toString(36).substr(2, 9);
22 }
23
24 // Route to get a new Math Captcha
25 app.get('/api/captcha', (req, res) => {
26   const num1 = Math.floor(Math.random() * 10) + 1;
27   const num2 = Math.floor(Math.random() * 10) + 1;
28
29   // Pick a random operator
30   const operators = ['+', '-', '*'];
31   const op = operators[Math.floor(Math.random() * operators.length)];
32
33   let answer;
34   let question;
35
36   if (op === '+') {
37     answer = num1 + num2;
38     question = `What is ${num1} + ${num2}?`;
39   } else if (op === '-') {
40     // Ensure result is non-negative
41     const max = Math.max(num1, num2);
42     const min = Math.min(num1, num2);
43     answer = max - min;
44     question = `What is ${max} - ${min}?`;
45   } else {
46     answer = num1 * num2;
47     question = `What is ${num1} * ${num2}?`;
48   }
49
50   const id = generateId();
51   captchas[id] = answer;
52
53   // Remove old captcha after 5 minutes to prevent memory leak
54   setTimeout(() => {
55     delete captchas[id];
56   }, 5 * 60 * 1000);
57
58   res.json({ question, answer });
59 });
60
61 app.listen(port, () => {
62   console.log(`Server is running on port ${port}`);
63 });
  
```

Results/Output:

Text Based

CAPTCHA

Type the text exactly as shown

Refresh



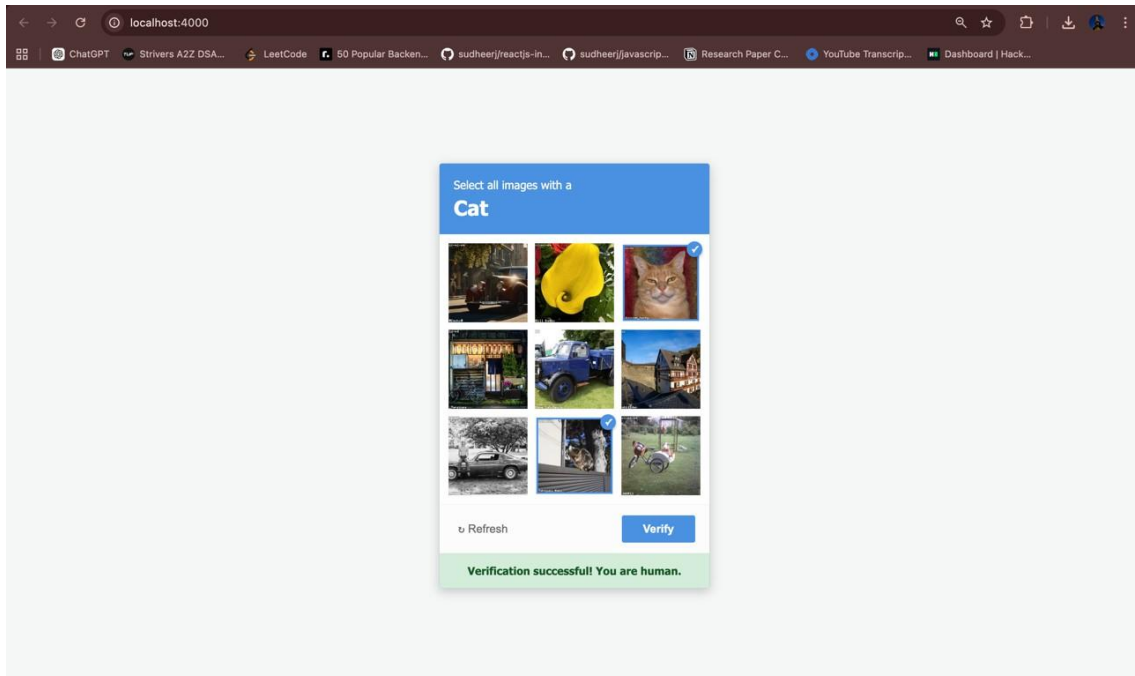
Enter CAPTCHA

Login

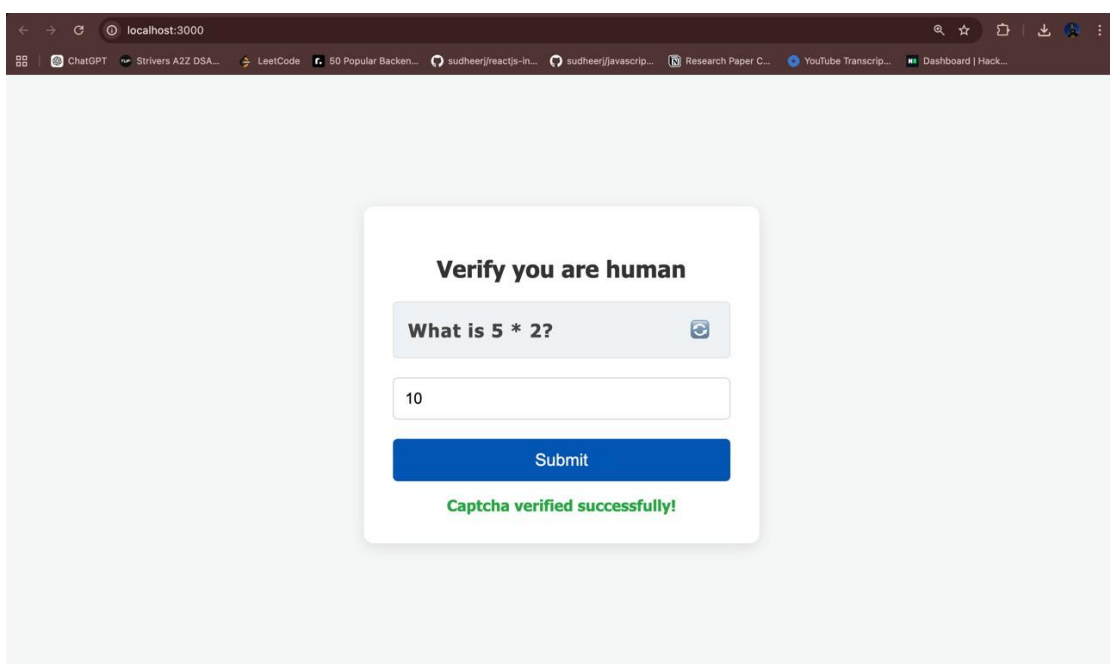
Logged in as admin

Somaiya Vidyavihar University K. J. Somaiya School of Engineering, Mumbai-77

Grid Image



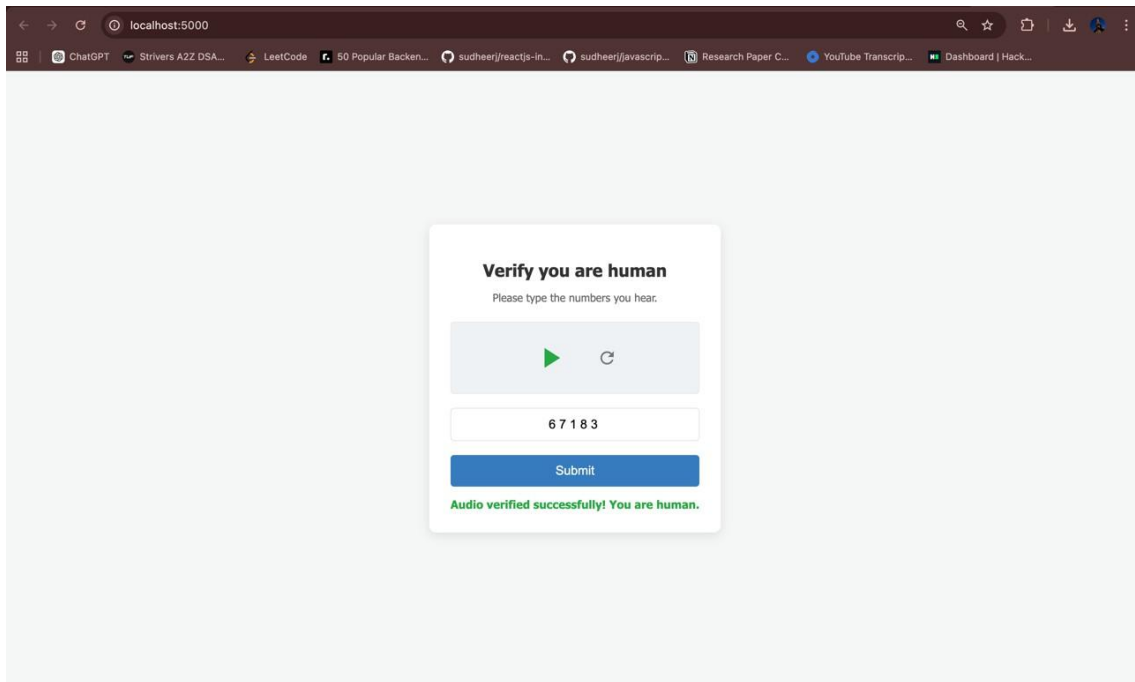
Mathematical Puzzle Based



Somaiya Vidyavihar University

K. J. Somaiya School of Engineering, Mumbai-77

Audio Based



Conclusion:

This project demonstrated how CAPTCHA systems use AI-hard problems like distorted text, audio, and logic challenges to differentiate humans from bots. It provided practical experience in generating and securely validating multi-modal CAPTCHAs. Overall, it emphasized balancing strong security with accessibility and user-friendly design.

Somaiya Vidyavihar University
K. J. Somaiya School of Engineering, Mumbai-77

Post-Lab Questions:

1. What is CAPTCHA and what security problem does it address?

CAPTCHA is a challenge-response mechanism that distinguishes human users from automated programs. It directly addresses the problem of bot-driven attacks such as spam submissions, credential stuffing, content scraping, and brute-force login attempts all of which can compromise the integrity and availability of web services.

2. Where is CAPTCHA typically deployed?

CAPTCHA is commonly integrated at high-risk interaction points within web applications, including login forms, account registration pages, password reset flows, online polls, comment sections, and checkout pages essentially anywhere automated abuse could cause significant harm.

3. What role does randomness play in CAPTCHA generation? Randomness is fundamental to CAPTCHA effectiveness. By generating unpredictable challenges each time, it ensures that bots cannot pre-compute answers, reuse previous responses, or exploit patterns to bypass the system. Without randomness, a CAPTCHA would quickly become predictable and trivially solvable.

4. Why is CAPTCHA considered a preventive security control?

CAPTCHA acts proactively rather than reactively it intercepts and blocks unauthorized automated access before any damage occurs, rather than detecting or responding to an attack after the fact. This places it firmly in the category of preventive controls within a layered security strategy.

5. Why must CAPTCHA validation always be performed on the server side?

Client-side validation can be easily manipulated or completely bypassed by an attacker using browser developer tools or automated scripts. Performing validation on the server ensures that the verification logic remains inaccessible and tamper-proof, making the challenge genuinely enforceable.

6. What happens when CAPTCHA validation fails repeatedly?

Repeated CAPTCHA failures typically trigger escalating security responses. The system may temporarily lock the user out, introduce time-based cooldowns, require more complex challenges, or flag the session for further review all aimed at discouraging persistent automated attempts while minimizing disruption to legitimate users.